TRƯỜNG ĐẠI HỌC
SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
HCMC University of Technology and Education
HCMUTE

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

CÔNG NGHỆ PHẦN MỀM MỚI (MTSE431179)

Node.js Platform

THS. NGUYỄN HỮU TRUNG

Thông tin Giảng viên

- Ths. Nguyễn Hữu Trung
- Khoa Công Nghệ Thông Tin
- Trường Đại học Sư Phạm Kỹ Thuật TP.HCM
- 090.861.7108
- trungnh@hcmute.edu.vn
- https://www.youtube.com/@baigiai

Nội dung bài học

- Node.js là gì
- Nguyên tắc hoạt động của Node.js
- Ưu và nhược điểm của Node.js
- Hướng dẫn cài đặt Node.js
- Những ứng dụng của Node.js
- Những Framework của Node.js

Node.js là gì

- Node.js được phát triển từ Javascript vào năm 2009 bởi Ryan Dahl. Node.js có cách thức hoạt động chủ yếu trên Server sử dụng để xây dựng cho các ứng dụng realtime. Node.js dùng mô hình I/O, lập trình dựa theo sự kiện non-blocking.
- Chính vì vậy, Node.js tương đối gọn nhẹ, hiệu quả và là một công cụ hoàn hảo dành cho mọi ứng dụng chuyên sâu về dữ liệu dựa theo khoảng thời gian thực khi chạy trên những thiết bị phân tán. Node.js thường xuyên được dùng cho mục đích xây dựng một số ứng dụng như: Ad Server, Websocket server, Fast File Upload Client, RESTful API, Cloud Services, Any Real-time Data Application.
- Node.js là một nền tảng phát triển phía server được xây dựng trên nền tảng JavaScript.

Nguyên tắc hoạt động của Node.js

- Khi một yêu cầu mạng đến từ một client, Node.js sẽ xử lý yêu cầu đó bằng cách thực hiện các bước như sau:
  - Node.js tạo một event loop để theo dõi các yêu cầu mạng đến và đi.
  - Khi một yêu cầu mạng đến, Node.js sẽ tạo một worker thread (luồng làm việc) để xử lý yêu cầu đó.
  - Trong worker thread, Node.js sẽ thực hiện các tác vụ xử lý yêu cầu, chẳng hạn như đọc và ghi vào cơ sở dữ liệu, đọc và ghi file, tương tác với API, ...
  - Khi worker thread đã hoàn thành các tác vụ, Node.js sẽ trả về kết quả cho client qua mạng.
  - Nếu có yêu cầu mạng mới đến, Node.js sẽ tạo một worker thread mới để xử lý yêu cầu đó.
- Các yêu cầu mạng đến và đi trong Node.js được xử lý bằng cách sử dụng các hàm callback, Promise, async/await để đảm bảo tính bất đồng bộ và tăng hiệu suất của ứng dụng. Các yêu cầu mạng được xử lý một cách độc lập, giúp tránh tình trạng "blocking" (chặn) trong quá trình xử lý yêu cầu. Node.js cũng có thể hoạt động với các module và thư viện khác để hỗ trợ cho việc phát triển ứng dụng web.

Ưu điểm của Node.js

- Hiệu suất cao: được thiết kế để xử lý các yêu cầu mạng phức tạp với tốc độ nhanh và hiệu quả.
- Lập trình bất đồng bộ (asynchronous programming): Node.js sử dụng các callback, Promise, async/await để xử lý các yêu cầu mạng phi đồng bộ, giúp tăng hiệu suất của ứng dụng và giảm thiểu thời gian đáp ứng cho người dùng.
- Có nhiều thư viện và module hỗ trợ: Node.js có rất nhiều thư viện và module hỗ trợ cho việc phát triển ứng dụng web, giúp các nhà phát triển nhanh chóng xây dựng ứng dụng với đầy đủ tính năng.
- Dễ dàng mở rộng: Node.js có khả năng mở rộng tốt, cho phép các nhà phát triển tăng hiệu suất của ứng dụng bằng cách thêm các máy chủ và máy chủ công việc (worker servers).
- Đơn giản và dễ học: Vì Node.js được xây dựng trên nền tảng JavaScript, nên các nhà phát triển có thể nhanh chóng học và sử dụng nó.

Nhược điểm của Node.js

- Không phù hợp cho các ứng dụng CPU-bound: Node.js thường được sử dụng cho các ứng dụng I/O-bound (yêu cầu đọc và ghi vào cơ sở dữ liệu, đọc và ghi file, tương tác với API, ...) nên nó không phù hợp cho các ứng dụng CPU-bound (như xử lý ảnh, phân tích dữ liệu lớn, ...).
- Có thể gây ra vấn đề khi xử lý lỗi: Khi xử lý lỗi trong Node.js, các nhà phát triển cần quản lý callback và exception để tránh gây ra vấn đề trong quá trình xử lý các yêu cầu mạng.
- Phiên bản thay đổi liên tục: Node.js thường có các phiên bản thay đổi liên tục, điều này đôi khi gây khó khăn cho các nhà phát triển khi cập nhật các thư viện và module để đảm bảo tính tương thích của ứng dụng.

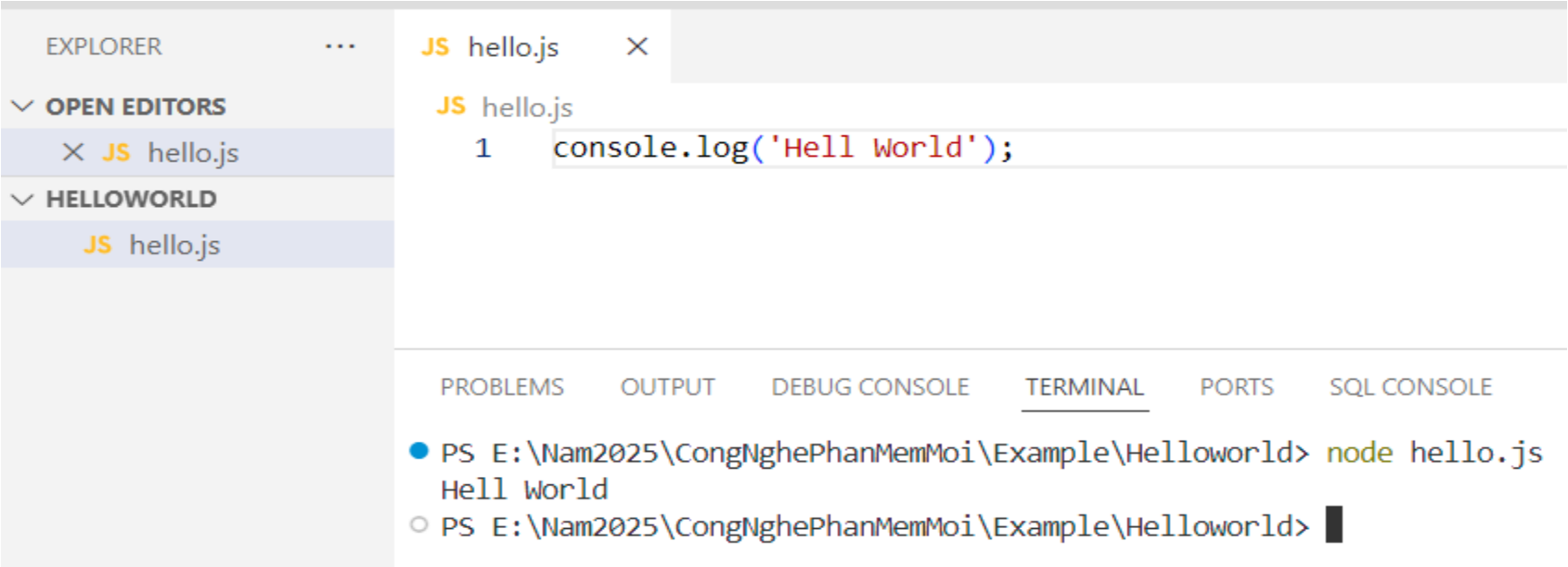
Những ứng dụng của Node.js

- Dynamic web applications: Node.js cho phép phát triển các ứng dụng web động như các trang web tương tác cao hoặc các ứng dụng phức tạp, cung cấp tính năng xử lý yêu cầu đồng thời và xử lý dữ liệu theo thời gian thực.
- Real-time web applications: Node.js cung cấp tính năng xử lý sự kiện theo thời gian thực, giúp phát triển các ứng dụng web thời gian thực như các ứng dụng chat, các trò chơi trực tuyến hoặc các ứng dụng tương tác cao.
- Xử lý các yêu cầu API (APIs): Node.js là một nền tảng phát triển tuyệt vời để xử lý các yêu cầu API, cho phép phát triển các ứng dụng web dựa trên API.
- Xử lý các tác vụ đồng bộ và không đồng bộ (synchronous và asynchronous tasks): Node.js cho phép phát triển các ứng dụng web đồng bộ và không đồng bộ, giúp tăng tốc độ xử lý của ứng dụng.
- Single-page applications: Node.js cung cấp các thư viện và framework như Express.js, Sails.js, Meteor.js để phát triển các ứng dụng web đơn trang.
- Microservices: Node.js cho phép phát triển các ứng dụng web liên kết với nhiều hệ thống khác nhau như cơ sở dữ liệu, các hệ thống phân tán, các dịch vụ bên thứ ba, giúp phát triển các ứng dụng web phức tạp và có tính mở rộng cao.

Cài đặt Node.js

- **Bước 1:** Vào website chính thức của Node.js để download và cài đặt. Node.js version mới nhất tại thời điểm viết bài là 22.0.0.
- **Bước 2:** Kiểm tra trạng thái cài đặt bằng cách nhập lệnh **node -v** vào command prompt.
- **Bước 3:** Tạo một folder để chứa các nội dung cho dự án. Trong đó tạo một file có tên *Project.js* rồi nhập nội dung cần viết:

```
console.log('Hell World');
```



Cài đặt nhiều phiên bản Node.js trên máy

- Bước 1: Uninstall node.js nếu đã có cài Node.js
- Bước 2: Install nvm: https://github.com/coreybutler/nvm-windows/releases
  - Hướng dẫn sử dụng: https://github.com/coreybutler/nvm-windows#readme
- Bước 3: Install node.js với phiên bản tương ứng:

  nvm install 18.20.3
- Bước 4: Nếu muốn sử dụng phiên bản nào thì dùng:

  nvm use 20.15.0
- Các lệnh kiểm tra:

  Node --version

  nvm --version

  nvm list

Ví dụ: Hello world

EXPLORER

OPEN EDITORS
- index.js

HELLOWORLD
- hello.js
- index.js

index.js > ...

```js
var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SQL CONSOLE

```
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> node hello.js
Hell World
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld>
* History restored

* History restored

PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> node index.js
```

localhost:8080

Android RecyclerVie... Thực hành Deploy d... Khoá

Hello World!

Node.js Modules

- Modules trong Node.js là một tập hợp các chức năng bạn muốn đưa vào ứng dụng của mình.
- Node.js có một bộ các modules tích hợp mà bạn có thể sử dụng mà không cần cài đặt thêm.
- Để thêm một module, bạn sử dụng hàm **require()** có tham số là tên modules

```
var http = require('http');
```

Bây giờ ứng dụng của bạn có quyền truy cập vào mô-đun HTTP và có thể tạo máy chủ:

```
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

Node.js Module

Module	Description
assert	Provides a set of assertion tests
buffer	To handle binary data
child_process	To run a child process
cluster	To split a single Node process into multiple processes
crypto	To handle OpenSSL cryptographic functions
dgram	Provides implementation of UDP datagram sockets
dns	To do DNS lookups and name resolution functions
events	To handle events

Node.js Module

fs	To handle the file system
http	To make Node.js act as an HTTP server
https	To make Node.js act as an HTTPS server.
net	To create servers and clients
os	Provides information about the operation system
path	To handle file paths
querystring	To handle URL query strings
readline	To handle readable streams one line at the time
stream	To handle streaming data
string_decoder	To decode buffer objects into strings

Node.js Module

timers	To execute a function after a given number of milliseconds
tls	To implement TLS and SSL protocols
tty	Provides classes used by a text terminal
url	To parse URL strings
util	To access utility functions
v8	To access information about V8 (the JavaScript engine)
vm	To compile JavaScript code in a virtual machine
zlib	To compress or decompress files

Tạo Modules

- Bạn có thể tạo các mô-đun của riêng mình và dễ dàng đưa chúng vào ứng dụng.
- Ví dụ sau đây tạo một mô-đun trả về một đối tượng ngày và giờ:

```
exports.NgayGio = function () {
  return Date();
};
```

Sử dụng **exports** để cung cấp các thuộc tính và phương thức bên ngoài tệp modules. Lưu mã ở trên vào tệp có tên "mymodule.js“. Bây giờ bạn có thể đưa vào và sử dụng module này trong bất kỳ tệp Node.js nào của mình.

```
var http = require('http');
var dt = require('./mymodule');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write("The date and time are currently:
" + dt.NgayGio());
  res.end();
}).listen(8080);
```

NPM - Node Package Manager

- NPM chính là công cụ quản lý thư viện của Node.js. Trang chủ của npm: https://www.npmjs.com/
- NPM cung cấp giao diện dòng lệnh (CLI) để chúng ta có thể cài đặt các thư viện cần thiết một cách dễ dàng.
- Tài liệu: https://docs.npmjs.com/cli/v8/commands/npm-install
- Câu lệnh sử dụng nhiều nhất: **npm i library_name**
- Hoặc viết đầy đủ là: **npm install library_name**
- Cài 1 package với version cụ thể:

npm install <package name>@<version>

Ví dụ: **npm i --save –exact express@4.1 8.2**

- Chạy nhiệm vụ: **npm run <task-name>**

```
{
  "scripts": {
    "start-dev": "node server-development.js",
    "start": "node server-production.js"
  }
}
```

$ npm run start

Câu lệnh **npm init**

- **npm init** giúp tạo ra file package.json và các thông tin cần thiết: app name, version,...

```
Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (helloworld)
version: (1.0.0)
description: Hello World
entry point: (index.js)
test command:
git repository:
keywords:
author: Nguyen Huu Trung
license: (ISC)
About to write to E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld\package.json:

{
  "name": "helloworld",
  "version": "1.0.0",
  "description": "Hello World",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Nguyen Huu Trung",
  "license": "ISC"
}


Is this OK? (yes) yes
```

package.json X

package.json > license

```
1  {
2    "name": "helloworld",
3    "version": "1.0.0",
4    "description": "Hello World",
5    "main": "index.js",
       Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "node index.js"
9    },
10   "author": "Nguyen Huu Trung",
11   "license": "ISC"
12 }
13
```

Commit Github


```
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> npm run test
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git add .
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git commit -m 'init project'
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'trungnh@DESKTOP-T2CV8JR.(none)')
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git config --global user.email "trungnh@hcmute.edu.vn"
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git commit -m 'init project'
[master (root-commit) 6055c7b] init project
 4 files changed, 22 insertions(+)
 create mode 100644 hello.js
 create mode 100644 index.js
 create mode 100644 myModule.js
 create mode 100644 package.json
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git remote add origin https://github.com/trungnhspkt2024/nodejs-reactjs.git
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git config --global user.name "trungnhspkt2024"
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git remote add origin https://github.com/trungnhspkt2024/nodejs-reactjs.git
error: remote origin already exists.
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git remote add origin git@github.com:trungnhspkt2024/nodejs-reactjs.git
error: remote origin already exists.
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git branch -M main
PS E:\Nam2025\CongNghePhanMemMoi\Example\Helloworld> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
```

Node.js HTTP Module

- Node.js có một mô-đun tích hợp có tên là HTTP, cho phép Node.js truyền dữ liệu qua Giao thức truyền siêu văn bản (HTTP).
- Để đưa mô-đun HTTP vào, hãy sử dụng require() :

```
var http = require('http');
```

- Mô-đun HTTP có thể tạo một máy chủ HTTP để lắng nghe các cổng máy chủ và phản hồi lại cho máy khách.
- Sử dụng **createServer()** này để tạo máy chủ HTTP:

```
var http = require('http');

//create a server object:
http.createServer(function (req, res) {
 res.writeHead(200, {'Content-Type': 'text/html’});// add header
  res.write('Hello World!'); //write a response to the client
  res.end(); //end the response
}).listen(8080); //the server object listens on port 8080
```

Node.js File System Module

- Cho phép bạn làm việc với hệ thống tệp trên máy tính của mình

```
var fs = require('fs');
```

- Công dụng phổ biến của mô-đun Hệ thống tập tin:
  - Đọc tập tin: `fs.readFile()`

```
var http = require('http');
var fs = require('fs');
http.createServer(function (req, res) {
  fs.readFile('demofile1.html', function(err, data) {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write(data);
    return res.end();
  });
}).listen(8080);
```

Node.js File System Module

- Cho phép bạn làm việc với hệ thống tệp trên máy tính của mình

```
var fs = require('fs');
```

- Công dụng phổ biến của mô-đun Hệ thống tập tin:
  - Tạo tập tin: fs.appendFile()

```
var fs = require('fs');

fs.appendFile('mynewfile1.txt', 'Hello content!', function (err) {
  if (err) throw err;
  console.log('Saved!');
});
```

Node.js File System Module

- Cho phép bạn làm việc với hệ thống tệp trên máy tính của mình

```
var fs = require('fs');
```

- Công dụng phổ biến của mô-đun Hệ thống tập tin:
  - Mở tập tin: fs.open()

```
var fs = require('fs');

fs.open('mynewfile2.txt', 'w', function (err, file) {
  if (err) throw err;
  console.log('Saved!');
});
```

Node.js File System Module

- Cho phép bạn làm việc với hệ thống tệp trên máy tính của mình

```
var fs = require('fs');
```

- Công dụng phổ biến của mô-đun Hệ thống tập tin:
  - Ghi tập tin: **fs.writeFile()**

```
var fs = require('fs');

fs.writeFile('mynewfile3.txt', 'Hello content!', function (err) {
  if (err) throw err;
  console.log('Saved!');
});
```

```
var fs = require('fs');

fs.rename('mynewfile1.txt', 'myrenamedfile.txt', function (err) {
  if (err) throw err;
  console.log('File Renamed!');
});
```

Đổi tên file

Node.js File System Module

- Cho phép bạn làm việc với hệ thống tệp trên máy tính của mình

```
var fs = require('fs');
```

- Công dụng phổ biến của mô-đun Hệ thống tập tin:
  - Cập nhật tập tin: fs.appendFile()

```
var fs = require('fs');

fs.appendFile('mynewfile1.txt', 'Hello content!', function (err) {
  if (err) throw err;
  console.log('Saved!');
});
```

Xóa file

```
var fs = require('fs');

fs.unlink('mynewfile2.txt', function (err) {
  if (err) throw err;
  console.log('File deleted!');
});
```

Node.js URL Module

- Mô-đun URL chia địa chỉ web thành các phần dễ đọc

```
var url = require('url');
```

```
var url = require('url');
var adr
= 'http://localhost:8080/default.htm?year=2024&month=july';
var q = url.parse(adr, true);

console.log(q.host); //returns 'localhost:8080'
console.log(q.pathname); //returns '/default.htm'
console.log(q.search); //returns '?year=2017&month=february'

var qdata = q.query; //returns an object: { year: 2017, month:
'february' }
console.log(qdata.month); //returns 'february'
```

Ví dụ: In hoa nội dung

- Bước 1: cài đặt module: **npm install upper-case**
- Bước 2: Sử dụng module: **var uc = require('upper-case');**

```
var http = require('http');
var uc = require('upper-case');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write(uc.upperCase("Hello World!")); //in hoa chữ lên
  res.end();
}).listen(8080);
```

Node.js Events


```
var fs = require('fs');
var rs = fs.createReadStream('./demofile.txt');
rs.on('open', function () {
  console.log('The file is open');
});
```

```
var events = require('events');
var eventEmitter = new events.EventEmitter();
```

```
var events = require('events');
var eventEmitter = new events.EventEmitter();

//Create an event handler:
var myEventHandler = function () {
  console.log('I hear a scream!');
}

//Assign the event handler to an event:
eventEmitter.on('scream', myEventHandler);

//Fire the 'scream' event:
eventEmitter.emit('scream');
```

Node.js Upload Files

- **npm install formidable**
- **var formidable = require('formidable’);**
- Bước 1: Tạo biểu mẫu tải lên

```
var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write('<form action="fileupload" method="post" enctype="multipart/form-data">');
  res.write('<input type="file" name="filetoupload"><br>');
  res.write('<input type="submit">');
  res.write('</form>');
  return res.end();
}).listen(8080);
```

Node.js Upload Files

- Bước 2: Phân tích tệp đã tải lên

```
var http = require('http');
var formidable = require('formidable');

http.createServer(function (req, res) {
  if (req.url == '/fileupload') {
    var form = new formidable.IncomingForm();
    form.parse(req, function (err, fields, files) {
      res.write('File uploaded');
      res.end();
    });
  } else {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write('<form action="fileupload" method="post" enctype="multipart/form-data">');
    res.write('<input type="file" name="filetoupload"><br>');
    res.write('<input type="submit">');
    res.write('</form>');
    return res.end();
  }
}).listen(8080);
```

Node.js Upload Files

Bước 3: Lưu tệp

```
var http = require('http');
var formidable = require('formidable');
var fs = require('fs');

http.createServer(function (req, res) {
  if (req.url == '/fileupload') {
    var form = new formidable.IncomingForm();
    form.parse(req, function (err, fields, files) {
      var oldpath = files.filetoupload.filepath;
      var newpath = 'C:/Users/Your Name/' + files.filetoupload.originalFilename;
      fs.rename(oldpath, newpath, function (err) {
        if (err) throw err;
        res.write('File uploaded and moved!');
        res.end();
      });
  });
  } else {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write('<form action="fileupload" method="post" enctype="multipart/form-data">');
    res.write('<input type="file" name="filetoupload"><br>');
    res.write('<input type="submit">');
    res.write('</form>');
    return res.end();
  }
}).listen(8080);
```

Node.js Send an Email

- npm install nodemailer
- var nodemailer = require('nodemailer');

```
var nodemailer = require('nodemailer');
var transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: 'youremail@gmail.com',
    pass: 'yourpassword'
  }
});
var mailOptions = {
  from: 'youremail@gmail.com',
  to: 'myfriend@yahoo.com',
  subject: 'Sending Email using Node.js',
  text: 'That was easy!'
};
transporter.sendMail(mailOptions, function(error, info){
  if (error) {
    console.log(error);
  } else {
    console.log('Email sent: ' + info.response);
  }
});
```

Node.js Send an Email

Multiple Receivers

```
var mailOptions = {
  from: 'youremail@gmail.com',
  to: 'myfriend@yahoo.com, myotherfriend@yahoo.com',
  subject: 'Sending Email using Node.js',
  text: 'That was easy!'
}
```

Send HTML

```
var mailOptions = {
  from: 'youremail@gmail.com',
  to: 'myfriend@yahoo.com',
  subject: 'Sending Email using Node.js',
  html: '<h1>Welcome</h1><p>That was easy!</p>'
}
```

Node.js MySQL

- `npm install mysql`
- `var mysql = require('mysql');`

```
var mysql = require('mysql');

var con = mysql.createConnection({
  host: "localhost",
  user: "yourusername",
  password: "yourpassword"
});

con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
});
```

- Query a Database

```
con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
  con.query(sql, function (err, result) {
    if (err) throw err;
    console.log("Result: " + result);
  });
});
```

Creating a Database

```
con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
  con.query("CREATE DATABASE mydb", function (err, result) {
    if (err) throw err;
    console.log("Database created");
  });
});
```

Creating a Table

```
var mysql = require('mysql');

var con = mysql.createConnection({
  host: "localhost",
  user: "yourusername",
  password: "yourpassword",
  database: "mydb"
});

con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
  var sql = "CREATE TABLE customers (name VARCHAR(255), address
VARCHAR(255))";
  con.query(sql, function (err, result) {
    if (err) throw err;
    console.log("Table created");
  });
});
```

Creating a Table

```
var mysql = require('mysql');

var con = mysql.createConnection({
  host: "localhost",
  user: "yourusername",
  password: "yourpassword",
  database: "mydb"
});
con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
  var sql = "CREATE TABLE customers (name VARCHAR(255), address
VARCHAR(255))";
  con.query(sql, function (err, result) {
    if (err) throw err;
    console.log("Table created");
  });
});
```

var sql = "CREATE TABLE customers (**id INT AUTO_INCREMENT PRIMARY KEY**, name VARCHAR(255), address VARCHAR(255))";

var sql = "ALTER TABLE customers ADD COLUMN id INT AUTO_INCREMENT PRIMARY KEY";

Insert Into Table

```
var mysql = require('mysql');

var con = mysql.createConnection({
  host: "localhost",
  user: "yourusername",
  password: "yourpassword",
  database: "mydb"
});

con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
  var sql = "INSERT INTO customers (name, address) VALUES ('Company Inc', 'Highway 37')";
  con.query(sql, function (err, result) {
    if (err) throw err;
    console.log("1 record inserted");
  });
});
```

Insert Multiple Records

```
con.connect(function(err) {
  if (err) throw err;
  console.log("Connected!");
  var sql = "INSERT INTO customers (name, address) VALUES ?";
  var values = [
    ['John', 'Highway 71'],
    ['Peter', 'Lowstreet 4'],
    ['Amy', 'Apple st 652'],
    ['Hannah', 'Mountain 21'],
    ['Michael', 'Valley 345'],
    ['Sandy', 'Ocean blvd 2'],
    ['Betty', 'Green Grass 1'],
    ['Richard', 'Sky st 331'],
    ['Susan', 'One way 98'],
    ['Vicky', 'Yellow Garden 2'],
    ['Ben', 'Park Lane 38'],
    ['William', 'Central st 954'],
    ['Chuck', 'Main Road 989'],
    ['Viola', 'Sideway 1633']
  ];
  con.query(sql, [values], function (err, result) {
    if (err) throw err;
    console.log("Number of records inserted: " + result.affectedRows);
  });
});
```

Selecting From a Table

```
con.connect(function(err) {
  if (err) throw err;
  con.query("SELECT * FROM customers", function (err, result, fields) {
    if (err) throw err;
    console.log(result);
  });
});
```

```
con.connect(function(err) {
  if (err) throw err;
  con.query("SELECT name, address FROM customers", function (err, result,
fields) {
    if (err) throw err;
    console.log(result);
  });
});
```

Selecting Columns

Select With a Filter

```
con.connect(function(err) {
  if (err) throw err;
  con.query("SELECT * FROM customers WHERE address = 'Park Lane 38'", function (err,
result) {
    if (err) throw err;
    console.log(result);
  });
});
```

```
con.connect(function(err) {
  if (err) throw err;
  con.query("SELECT * FROM customers WHERE address LIKE 'S%'", function (err,
result) {
    if (err) throw err;
    console.log(result);
  });
});
```

Wildcard Characters

Escaping Query Values

```
var adr = 'Mountain 21';
var sql = 'SELECT * FROM customers WHERE address = ' + mysql.escape(adr);
con.query(sql, function (err, result) {
  if (err) throw err;
  console.log(result);
});
```

```
var adr = 'Mountain 21';
var sql = 'SELECT * FROM customers WHERE address = ?';
con.query(sql, [adr], function (err, result) {
  if (err) throw err;
  console.log(result);
});
```

```
var name = 'Amy';
var adr = 'Mountain 21';
var sql = 'SELECT * FROM customers WHERE name = ? OR address = ?';
con.query(sql, [name, adr], function (err, result) {
  if (err) throw err;
  console.log(result);
});
```

Sort the Result

```
con.connect(function(err) {
  if (err) throw err;
  con.query("SELECT * FROM customers ORDER BY name", function (err, result) {
    if (err) throw err;
    console.log(result);
  });
});
```

```
con.connect(function(err) {
  if (err) throw err;
  con.query("SELECT * FROM customers ORDER BY name DESC", function (err, result) {
    if (err) throw err;
    console.log(result);
  });
});
```

Delete Record

```
con.connect(function(err) {
  if (err) throw err;
  var sql = "DELETE FROM customers WHERE address = 'Mountain 21'";
  con.query(sql, function (err, result) {
    if (err) throw err;
    console.log("Number of records deleted: " + result.affectedRows);
  });
});
```

Delete a Table

```
con.connect(function(err) {
  if (err) throw err;
  var sql = "DROP TABLE customers";
  con.query(sql, function (err, result) {
    if (err) throw err;
    console.log("Table deleted");
  });
});
```

```
var sql = "DROP TABLE IF EXISTS customers";
```

Update Table

```
con.connect(function(err) {
  if (err) throw err;
  var sql = "UPDATE customers SET address = 'Canyon 123' WHERE address = 'Valley 345'";
  con.query(sql, function (err, result) {
    if (err) throw err;
    console.log(result.affectedRows + " record(s) updated");
  });
});
```

```
con.connect(function(err) {
  if (err) throw err;
  var sql = "SELECT * FROM customers LIMIT 5";
  con.query(sql, function (err, result) {
    if (err) throw err;
    console.log(result);
  });
});
```

Limit the Result

```
var sql = "SELECT * FROM customers LIMIT 5 OFFSET 2";
```

Join Two or More Tables

```
con.connect(function(err) {
  if (err) throw err;
  var sql = "SELECT users.name AS user, products.name AS favorite
FROM users JOIN products ON users.favorite_product = products.id";
  con.query(sql, function (err, result) {
    if (err) throw err;
    console.log(result);
  });
});
```

- Nếu bạn muốn trả về *tất cả* người dùng, bất kể họ có sản phẩm yêu thích hay không, hãy sử dụng câu lệnh LEFT JOIN:
  - **LEFT JOIN products ON users.favorite_product = products.id**

- Nếu bạn muốn trả về tất cả các sản phẩm và những người dùng coi chúng là sản phẩm yêu thích, ngay cả khi không có người dùng nào coi chúng là sản phẩm yêu thích, hãy sử dụng câu lệnh RIGHT JOIN
  - **RIGHT JOIN products ON users.favorite_product = products.id**

Node.js kết nối Mysql bằng sequelize


```
import { Sequelize } from 'sequelize';
//const { Sequelize } = require('sequelize');//ES5 module

// Option 3: Passing parameters separately (other dialects)
const sequelize = new Sequelize('node_fulltask', 'root',
'1234567@a$', {
  host: 'localhost',
  dialect: 'mysql',
  logging: false
});
let connectDB = async () => {
    try {
        await sequelize.authenticate();
        console.log('Connection has been established successfully.');
    } catch (error) {
        console.error('Unable to connect to the database:', error);
    }
}
module.exports = connectDB;
```

Node.js MongoDB

- `npm install mongodb`
- `var mongo = require('mongodb’);`
- Creating a Database

```
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/mydb";

MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  console.log("Database created!");
  db.close();
});
```

Creating a Collection

```
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/";

MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  dbo.createCollection("customers", function(err, res) {
    if (err) throw err;
    console.log("Collection created!");
    db.close();
  });
});
```

Insert Into Collection

```
var MongoClient = require('mongodb').MongoClient;
var url = "mongodb://localhost:27017/";

MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  var myobj = { name: "Company Inc", address: "Highway 37" };
  dbo.collection("customers").insertOne(myobj, function(err, res)
{
    if (err) throw err;
    console.log("1 document inserted");
    db.close();
  });
});
```

Insert Multiple Documents

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  var myobj = [
    { name: 'John', address: 'Highway 71'},
    { name: 'Peter', address: 'Lowstreet 4'},
    { name: 'Betty', address: 'Green Grass 1'},
    { name: 'Viola', address: 'Sideway 1633'}
  ];
  dbo.collection("customers").insertMany(myobj, function(err, res)
{
    if (err) throw err;
    console.log("Number of documents inserted: " +
res.insertedCount);
    db.close();
  });
});
```

Find One

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  dbo.collection("customers").findOne({}, function(err, result) {
    if (err) throw err;
    console.log(result.name);
    db.close();
  });
});
```

Find All

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  dbo.collection("customers").find({}).toArray(function(err,
result) {
    if (err) throw err;
    console.log(result);
    db.close();
  });
});
```

```
dbo.collection("customers").find({
}, { projection: { _id: 0,
name: 1, address: 1 } })
```

Query

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  var query = { address: "Park Lane 38" };
  dbo.collection("customers").find(query).toArray(function(err, result) {
    if (err) throw err;
    console.log(result);
    db.close();
  });
});
```

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  var query = { address: /^S/ };
  dbo.collection("customers").find(query).toArray(function(err, result) {
    if (err) throw err;
    console.log(result);
    db.close();
  });
});
```

- Sort

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  var mysort = { name: 1 };
  dbo.collection("customers").find().sort(mysort).toArray(function
(err, result) {
    if (err) throw err;
    console.log(result);
    db.close();
  });
});
```

```
{ name: 1 } // ascending
{ name: -1 } // descending
```

Delete

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  var myquery = { address: 'Mountain 21' };
  dbo.collection("customers").deleteOne(myquery, function(err, obj) {
    if (err) throw err;
    console.log("1 document deleted");
    db.close();
  });
});
```

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  var myquery = { address: /^O/ };
  dbo.collection("customers").deleteMany(myquery, function(err, obj) {
    if (err) throw err;
    console.log(obj.result.n + " document(s) deleted");
    db.close();
  });
});
```

Drop

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  dbo.collection("customers").drop(function(err, delOK) {
    if (err) throw err;
    if (delOK) console.log("Collection deleted");
    db.close();
  });
});
```

Delete Table

```
dbo.dropCollection("customers", function(err, delOK)
```

Update

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  var myquery = { address: "Valley 345" };
  var newvalues = { $set: {name: "Mickey", address: "Canyon 123" } };
  dbo.collection("customers").updateOne(myquery, newvalues, function(err, res) {
    if (err) throw err;
    console.log("1 document updated");
    db.close();
  });
});
```

Update Only Specific Fields

```
var myquery = { address: "Valley 345" };
var newvalues = { $set: { address: "Canyon 123" } };
dbo.collection("customers").updateOne(myquery, newvalues, function(err, res) {
```

Update Many Documents

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  var myquery = { address: /^S/ };
  var newvalues = {$set: {name: "Minnie"} };
  dbo.collection("customers").updateMany(myquery,
newvalues, function(err, res) {
    if (err) throw err;
    console.log(res.result.nModified + " document(s) updated");
    db.close();
  });
});
```

Limit the Result

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  dbo.collection("customers").find().limit(5).toArray(function(err,
result) {
    if (err) throw err;
    console.log(result);
    db.close();
  });
});
```

Join Collections

```
MongoClient.connect(url, function(err, db) {
  if (err) throw err;
  var dbo = db.db("mydb");
  dbo.collection('orders').aggregate([
    { $lookup:
       {
         from: 'products',
         localField: 'product_id',
         foreignField: '_id',
         as: 'orderdetails'
       }
     }
    ]).toArray(function(err, res) {
    if (err) throw err;
    console.log(JSON.stringify(res));
    db.close();
  });
});
```

Node.js kết nối database Mongodb bằng Mongoose


```
import mongoose from 'mongoose';

export const connectDb = async () => {
  try {
    const options = {
      useNewUrlParser: true,
      useUnifiedTopology: true,
      minPoolSize: 1,
      maxPoolSize: 50,
    };
    await mongoose.connect(`${process.env.DATABASE_URL}`, options);

    console.log('MongoDB connected!');
  } catch (error) {
    console.error('Error connecting to MongoDB:', error);
    throw error;
  }
};
```

Với biến môi trường trong file .env

DATABASE_URL = mongodb://0.0.0.0:27017/

Framework của Node.js

- **Express.js:** framework này linh hoạt và đơn giản hóa được thiết kế đặc biệt để xây dựng các ứng dụng một trang, nhiều trang và cung cấp một bộ tính năng phong phú hỗ trợ cho các ứng dụng web và di động
- **Nest.js:** framework này đang được phát triển, giúp xây dựng các ứng dụng server-side hiệu quả, đáng tin cậy và có khả năng mở rộng. Framework này giúp các nhà lập trình xây dựng các ứng dụng web bên máy chủ theo mô-đun, có khả năng mở rộng và dễ bảo trì.
- **Hapi.js:** Frameworks này phù hợp để xây dựng các ứng dụng và hỗ trợ các giải pháp cho các hệ thống scale lớn như Paypal, Disney.
- **LoopBack.js**
- **Koa.js**
- **Sails.js**
- **Meteor.js**
- **Derby.js**
- **Total.js**
- **Adonis.js**